UNIFAAT 2026.2

08 - Paradigma Orientado a Objetos

Prof. Luan Lourenço

Do dado pronto ao dado que a gente desenha

Até aqui só usamos o que o Python traz pronto:

```
# dados prontos
int · float · bool · str
list · tuple · dict
não existe "aluno", nem "pasta"
```

```
# dados que a gente desenha
Aluno: nome, ra, notas
Item: nome, pai, filhos
o molde se chama classe
```

Uma árvore é feita de pedaços que apontam uns para os outros, e cada pedaço é um objeto. Por isso a orientação a objetos vem **antes** das estruturas de dados.

Função: entrada → saída

```
# parâmetros (entrada) → corpo → return (saída)
parâmetro: o nome que a função declara pra receber
argumento: o valor passado na hora de chamar
```

src/08/funcoes.py

```python
def calcular_media(primeira_nota, segunda_nota):  # parâmetros
    soma = primeira_nota + segunda_nota
    media = soma / 2
    return media  # a saída

resultado = calcular_media(8, 6)  # argumentos
print(resultado)
7.0
```

Na chamada, o 1º argumento entra no 1º parâmetro: **a ordem importa**.

`return` não é `print`

com return

```
def media(a, b):
    return (a + b) / 2

valor = media(8, 6)
print(valor + 1)
8.0
```

só print

```
def media(a, b):
    print((a + b) / 2)

valor = media(8, 6)
print(valor)
7.0
None
```

→ `return` **devolve** o valor: dá pra guardar, somar, comparar e encerra a função na hora

→ `print` só **mostra** na tela; quem chamou não recebe nada

Sem retorno: void e procedure

tipo	tem return?	exemplo
função	sim	`calcular_media(8, 6)` devolve `7.0`
procedimento (*procedure*)	**não: devolve** `None`	`mostrar_boletim("Ana", 7.0)`
sem parâmetros	tanto faz	`mostrar_cabecalho()`

src/08/funcoes.py

```
def mostrar_boletim(nome, media):
    print("Aluno:", nome, "| Média:", media)  # só faz, não devolve

def aprovado(media, media_minima=7):  # valor padrão
    return media >= media_minima
```

Em C, sem retorno é `void`. Em Python não existe a palavra: função sem `return` devolve `None` ("nada").

O que é um objeto?

Uma **coisa do problema** representada no código: ela **tem** dados e **sabe fazer** coisas com eles.

```
# no mundo: um aluno
tem: nome, RA, notas
faz: recebe nota,
      calcula a média
```

```
# no código: o objeto ana
dados: "Ana", 1234, [8, 9]
comportamento:
  adicionar_nota()
  calcular_media()
```

Antes: dados soltos e funções soltas. Agora: **dados e o que se faz com eles, juntos num lugar só**. Um `int` ou uma `str` não sabem nada sobre alunos; o objeto sabe.

Orientar a objetos é mais que classe e objeto

Classe e objeto são a **ferramenta**. Saber usar a ferramenta não basta: orientação a objetos é uma **filosofia** de organizar o código.

```
# a ferramenta (a sintaxe)
classe e objeto
propriedades e métodos
self e construtor
é o que vemos hoje
```

```
# a filosofia (os pilares)
abstração
encapsulamento
herança
polimorfismo
são 3 ou 4? o que cada nome seria?
```

Escrever uma classe não é orientar a objetos, do mesmo jeito que saber usar o martelo não é saber construir uma casa. Os pilares ficam pra **próxima aula**.

Onde nasce o objeto: a classe

```
# C: só agrupa os dados
struct Aluno {
    char nome[50];
    int ra;
};
```

```
# Python: dados + funções
class Aluno:
    nome, ra, notas
    adicionar_nota()
    calcular_media()
```

	o que é	exemplo
classe	o molde: define como o dado vai ser	`Aluno`
objeto	o dado criado a partir do molde	`ana`, `bruno`

Do mesmo molde saem quantos objetos quisermos, cada um com os seus próprios dados.

O objeto agrupa propriedades e métodos

```
# propriedade = o que o objeto TEM
dado que pertence ao objeto
qualquer tipo:
  nome (str) · ra (int)
  notas (list) · até outro objeto
```

```
# método = o que o objeto FAZ
função que pertence ao objeto
mesmos parâmetros e return
do tópico de funções
chamada com ponto: ana.calcular_media()
```

Método é só uma função declarada **dentro da classe**.

O `self`: o objeto apontando pra si mesmo

src/08/classes.py

```
class Aluno:
    def adicionar_nota(self, nota):
        self.notas.append(nota)

    def calcular_media(self):
        soma = 0
        for nota in self.notas:
            soma = soma + nota
        return soma / len(self.notas)

ana = Aluno()
bruno = Aluno()
ana.adicionar_nota(8)
bruno.adicionar_nota(4)
```

```
# o Python entrega o objeto ao self
ana.adicionar_nota(8)
  → self = ana
  → mexe na lista da Ana

bruno.adicionar_nota(4)
  → self = bruno
  → mexe na lista do Bruno
```

O método é escrito uma vez e serve pra todos os objetos. Sem o `self`, ele perderia a referência de **de quem** é cada dado.

Construtor: `__init__`

Método especial, chamado **no instante em que o objeto é criado**. Os argumentos de `Aluno(...)` chegam nele.

sem construtor

```
ana = Aluno()
ana.nome = "Ana"
ana.ra = 1234
ana.notas = []
# esqueceu uma? AttributeError
```

com construtor

```
class Aluno:
    def __init__(self, nome, ra):
        self.nome = nome
        self.ra = ra
        self.notas = []

ana = Aluno("Ana", 1234)
```

O objeto já nasce completo. Em JavaScript é o `constructor`; em PHP, o `__construct`.

Propriedade pode ser outro objeto

src/08/construtor.py

```
class Professor:
    def __init__(self, nome):
        self.nome = nome

class Turma:
    def __init__(self, nome, professor):
        self.nome = nome
        self.professor = professor  # objeto injetado
        self.alunos = []

luan = Professor("Luan")
turma = Turma("ADS 2° semestre", luan)
print(turma.professor.nome)
Luan
```

A turma não copia o professor: guarda uma **referência** ao mesmo objeto. É essa ideia que monta a árvore.

Árvore de arquivos: cada item conhece o pai

```
# o que o Explorador mostra
C:/
    Users/
        Desktop/
            Trabalhos/
                tf-08.py
```

```
# classe Item
nome: "tf-08.py"
eh_pasta: False
pai: o objeto trabalhos
filhos: [ ] (só pasta tem)
```

Pasta e arquivo são o mesmo tipo de dado: a diferença é só o booleano `eh_pasta`. A raiz é o único item sem pai (`None`).

A classe Item

src/08/arvore_arquivos.py

```
class Item:
    def __init__(self, nome, eh_pasta, pai=None):
        # só pasta pode conter itens
        if pai is not None and not pai.eh_pasta:
            raise ValueError(pai.nome + " é um arquivo")

        self.nome = nome
        self.eh_pasta = eh_pasta
        self.pai = pai  # o objeto injetado
        self.filhos = []

        if pai is not None:
            pai.filhos.append(self)  # se cadastra no pai
```

O pai é injetado, então precisa **existir antes** do filho. Sem `else`: o `if` só age quando há pai.

Montando a árvore: primeiro o pai

src/08/arvore_arquivos.py

```
disco = Item("C:", True)
usuarios = Item("Users", True, disco)
desktop = Item("Desktop", True, usuarios)
trabalhos = Item("Trabalhos", True, desktop)
Item("notas.txt", False, desktop)
tf = Item("tf-08.py", False, trabalhos)
Item("slide.pdf", False, trabalhos)

disco.mostrar()
```

saída

```
C:/
    Users/
        Desktop/
            Trabalhos/
                tf-08.py
                slide.pdf
            notas.txt
```

Cada linha cria um objeto e injeta o pai no construtor.

Próximos passos

- A **filosofia** de orientar a objetos: os pilares
- Usar **objetos para montar estruturas de dados**: a árvore de arquivos foi só o começo

tarefa

TF 08: commit e repositório

Num repositório novo `tf-08`, com venv e `.gitignore`, criar no `commit.py` o histórico do Git em objetos:

- `Commit`: `id`, `mensagem`, `autor` e `pai` (o commit anterior); o `historico()` sobe de pai em pai
- `Repositorio`: `commitar()` (um contador gera o **id único**), `log()` e `total_de_commits()`
- Autor = **nome e RA** do aluno; 3+ commits com mensagens suas
- Sem `else`, **sem** `import`, comentários com as próprias palavras, `.venv` fora do repositório

Entrega: link do repositório público no GitHub, pelo formulário abaixo — `docs.google.com/forms/d/e/1FAIpQLSedMOT1GH_eZeXlU-WdvvE8HfQfe2he01c4yeqElX2tvn3jaw/viewform`

Prazo: **21/10, até 23:59**.